JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 11 OF 18

Stacks, Queues, Deques, and Monotonic Patterns

LIFO, FIFO, Sliding Extrema, Next-Greater State, and Java ArrayDeque

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Choose LIFO, FIFO, deque, priority, or monotonic state correctly and implement the pattern with safe Java collection APIs.

STACK | QUEUE | DEQUE | MONOTONIC | WINDOWS | ARRAYDEQUE

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `10 – Linked Lists`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume unifies ordering data structures around the state each one preserves, then applies monotonic structures to linear-time interview patterns.

Readiness map

Decision	Evidence
START HERE	The most recent unresolved item matters; Work must be processed in arrival order; Both ends need efficient updates; Dominated candidates can be discarded while preserving extrema.
PREPARE FIRST	Volumes 1-10; Array and linked-list fundamentals.
MOVE ON WHEN	Use ArrayDeque for stack and queue roles; Distinguish queue, deque, and priority semantics; Implement next-greater and sliding-window maximum; Explain amortized linear monotonic processing.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

10 - Linked Lists	**YOU ARE HERE** **11 - Stacks, Queues, and Deques**	12 - Binary Search

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Stacks, Queues, Deques, and Monotonic Patterns for SDE-2

These structures encode an ordering policy. A stack says the newest unfinished item is resolved first. A queue says work is resolved in discovery order. A deque permits controlled access at both boundaries. Monotonic structures add a dominance rule: discard entries that can never influence a future answer. SDE-2 candidates should recognize the policy from the problem, prove why discarded state is irrelevant, and connect the in-memory algorithm to bounded queues, load shedding, and backpressure in production.

Choose the policy before the class

Prompt signal	Policy	Java default
nested delimiters, undo, expression operators	LIFO	`ArrayDeque<E>` as stack
level order, shortest unweighted steps, arrival order	FIFO	`ArrayDeque<E>` as queue
both ends, window candidates, 0-1 transitions	deque	`ArrayDeque<E>`
nearest greater/smaller boundary	monotonic stack	deque of indexes
best element in each moving window	monotonic deque	deque of indexes
producer may outrun consumer	bounded blocking/nonblocking queue	explicit capacity and rejection policy

Prefer `ArrayDeque` over legacy `Stack`. It has no synchronization overhead, exposes both stack and queue operations, and rejects `null`, preventing ambiguity with methods that use `null` for "empty." It is not thread-safe.

The ArrayDeque API without end confusion

Intent	Mutating method	Empty/full alternative	End
stack push	`push(e)` / `addFirst(e)`	`offerFirst(e)`	front
stack pop	`pop()` / `removeFirst()`	`pollFirst()` returns `null`	front
stack peek	`getFirst()`	`peekFirst()` returns `null`	front
queue enqueue	`addLast(e)`	`offerLast(e)`	back
queue dequeue	`removeFirst()`	`pollFirst()` returns `null`	front
inspect queue head	`getFirst()`	`peekFirst()` returns `null`	front

`ArrayDeque` grows and therefore does not express operational capacity. Use `ArrayBlockingQueue`, another bounded queue, or a domain-specific ring buffer when overload must be visible. Method names do not provide thread safety or backpressure by themselves.

Family 1: delimiter validation

Recognition and invariant

Use a stack when every closing token must match the most recent unmatched opening token. Scan left to right. Before processing position `i`, the stack contains exactly the unmatched openings in the processed prefix, oldest at the bottom and newest at the top. Push openings. On a close, reject if the stack is empty or its top has the wrong type. At the end, accept only if the stack is empty.

Dry-run `{[()]}`: stack states are `{`, `{[`, `{[(`, `{[`, `{`, empty. For `([)]`, `)` sees `[` on top and fails even though some `(` exists deeper; nesting order, not just counts, is the requirement.

Time is `O(n)`, space is `O(n)` in the all-opening case. Define how non-delimiter characters are treated. The sample ignores them; a parser might reject them or tokenize strings/comments first. Never validate source code correctly by scanning raw characters without lexical rules.

Family 2: expression evaluation

Expression questions test precedence, associativity, tokenization, unary operators, and failure contracts. Two common designs are operator/value stacks (shunting-yard style) and recursive descent. The standalone implementation below uses recursive descent with this grammar:

TEXT

```
expression := term (("+" | "-") term)*
term       := factor (("*" | "/") factor)*
factor     := ("+" | "-") factor | number | "(" expression ")"
```

The sample grammar deliberately defines `number` as one or more ASCII digits `0` through `9`. Accepting broader Unicode digits requires digit-value conversion rather than subtracting the ASCII character `'0'`.

The invariant of `parseExpression` is that it enters at the beginning of an expression and returns its value with the cursor at the first token not belonging to that expression. `parseTerm` consumes multiplication/division before the expression layer sees addition/subtraction, encoding precedence structurally. The loops combine from left to right, giving left associativity for subtraction and division.

Dry-run `2 + 3 * (4 - 1)`: expression first obtains term `2`. It sees `+`; the next term obtains factor `3`, sees `*`, then evaluates parenthesized expression `4-1` as `3`. The term becomes `9`; the outer expression returns `11`.

Parsing is `O(n)` time and `O(d)` call depth for nesting/unary depth `d`. Arithmetic overflow, division rounding, numeric format, maximum nesting, and error location are API decisions. The sample uses `long`, Java's truncation toward zero, and explicit malformed-input errors. It detects overflow while accumulating a numeric literal, but the expression operators and unary negation intentionally retain ordinary Java `long` wraparound; a strict-arithmetic API would use the corresponding exact operations throughout.

Family 3: MinStack and synchronized representations

A MinStack supports `push`, `pop`, `top`, and `min` in constant time. One design stores pairs `(value, minimumSoFar)`. Another maintains a value stack plus a minimum stack. When pushing a value less than or equal to the current minimum, also push it on minima. On pop, remove from minima when the values equal.

The `<=` is important. For pushes `3,2,2`, minima must contain both copies of `2`; after one `2` is popped, the remaining minimum is still `2`. The invariant is that the minima stack holds exactly those values that established or tied a minimum among the corresponding value-stack prefixes. Operations are `O(1)` time and total space `O(n)`.

In concurrent code, two deques must be updated atomically under one lock or confined to one thread. Thread-safe components used separately do not create a thread-safe compound invariant.

Family 4: a circular queue

A fixed-size ring buffer makes capacity explicit. Track `head`, `size`, and an array. The tail insertion index is `(head + size) % capacity`. On removal, read `head`, advance it modulo capacity, and decrement size.

The invariant is that logical element `j`, for `0 <= j < size`, resides at `(head+j) % capacity`; all other slots are outside the logical queue. Keeping `size` distinguishes full from empty when head and tail indexes coincide. With capacity three: offer `10,20,30`; poll `10` moves head; offer `40` wraps into the freed slot, while logical order remains `20,30,40`.

Each operation is `O(1)`, space `O(capacity)`. Decide whether full means return `false`, throw, block, drop newest, drop oldest, or spill elsewhere. That policy is more important operationally than modulo arithmetic.

Family 5: monotonic stacks

Next greater and daily temperatures

For each element, a next-greater problem asks which later item first dominates it. Keep indexes whose answer is unresolved in decreasing value order. When current value exceeds the value at the top index, the current index is that older entry's first greater answer; pop and resolve repeatedly, then push current.

Why is it the first? Every intervening value failed to exceed the older value, or the older index would already have been popped. Why can it be removed? Its one requested answer is now final. Every index is pushed once and popped at most once, so the nested `while` is aggregate `O(n)`, not `O(n^2)`.

Dry-run temperatures `[73,74,75,71,69,72,76,73]`: index `0` resolves at `1`; index `1` resolves at `2`. Indexes `2,3,4` accumulate; value `72` resolves `4` and `3`, but not `2`. Value `76` resolves `5` and `2`. Waiting days become `[1,1,4,2,1,1,0,0]`.

Use `<` versus `<=` according to "greater" versus "greater or equal." Store indexes when distance, expiration, or duplicate identity matters.

Largest rectangle in a histogram

Maintain indexes with nondecreasing heights. When a lower height arrives, pop height `h`; after popping, the new stack top is the nearest strictly lower boundary on the left, and current index is the first lower boundary on the right. Width is `right - left - 1`. A virtual trailing zero flushes remaining bars.

For `[2,1,5,6,2,3]`, arrival of height `2` at index `4` pops `6` (area `6*1`) and `5` (area `5*2=10`). That is the optimum. Each index enters and leaves once: `O(n)` time and `O(n)` space. Equality policy changes which duplicate bar carries the width but should not change the maximum if implemented consistently.

Family 6: monotonic deque for sliding maximum

The deque stores candidate indexes in decreasing value order and increasing index order. Before producing window ending at `right`:

1. remove the front if it lies before `right-k+1`;
2. remove back indexes whose values are no greater than the new value;
3. append `right`;
4. the front is the maximum once a full window exists.

Removing dominated backs is sound: the new value is at least as large and expires later, so the old candidate can never again be a maximum. For `[1,3,-1,-3,5,3,6,7]`, `k=3`, maxima are `[3,3,5,5,6,7]`. Index storage is mandatory because values alone do not reveal expiration.

Time is `O(n)` aggregate, space `O(k)`. Validate `1 <= k <= n`. For immutable inputs, returning values is fine; a streaming system should define timestamp/window semantics and out-of-order handling.

Family 7: BFS frontier and operational backpressure

Breadth-first search uses a FIFO queue because all nodes at distance `d` must be expanded before any node at `d+1`. The invariant is that when a vertex is dequeued, its recorded distance is the shortest unweighted distance from the source. Mark visited when enqueuing, not when dequeuing; otherwise multiple parents can enqueue the same vertex and inflate memory.

On a grid, each edge changes one coordinate. Starting distance zero, enqueue legal unvisited neighbors with distance plus one. The first time the target is discovered or dequeued is shortest because FIFO order never skips a smaller layer. Complexity is `O(V+E)`; on an `r*c` four-neighbor grid, `O(r*c)` time and space.

An algorithmic frontier can grow to `O(V)`. A service queue can grow until the process fails. Production queues need capacity, admission control, timeouts, cancellation, prioritization, fairness, and metrics. "Use a queue" is not a complete system design.

Complete Java 21 reference implementation

Compile and run with `java -ea StackQueueDequeSde2`.

JAVA (continued 1/8)

```java
import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;
import java.util.NoSuchElementException;
import java.util.OptionalInt;

public final class StackQueueDequeSde2 {
    private StackQueueDequeSde2() {}

    public static boolean validDelimiters(String text) {
        if (text == null) throw new IllegalArgumentException("text is null");
        Deque<Character> openings = new ArrayDeque<>();
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch == '(' || ch == '[' || ch == '{') {
                openings.push(ch);
            } else if (ch == ')' || ch == ']' || ch == '}') {
                if (openings.isEmpty() || !matches(openings.pop(), ch)) return false;
            }
        }
        return openings.isEmpty();
    }

    private static boolean matches(char open, char close) {
        return open == '(' && close == ')'
                || open == '[' && close == ']'
                || open == '{' && close == '}';
    }

    public static long evaluate(String expression) {
        if (expression == null) throw new IllegalArgumentException("expression is null");
        Parser parser = new Parser(expression);
        long value = parser.expression();
        parser.spaces();
        if (!parser.done()) throw parser.error("unexpected token");
```

JAVA (continued 2/8)

```
        return value;
    }

    private static final class Parser {
        private final String input;
        private int at;
        Parser(String input) { this.input = input; }
        boolean done() { return at == input.length(); }
        IllegalArgumentException error(String message) {
            return new IllegalArgumentException(message + " at offset " + at);
        }
        void spaces() {
            while (!done() && Character.isWhitespace(input.charAt(at))) at++;
        }
        boolean take(char expected) {
            spaces();
            if (!done() && input.charAt(at) == expected) { at++; return true; }
            return false;
        }
        long expression() {
            long value = term();
            while (true) {
                if (take('+')) value += term();
                else if (take('-')) value -= term();
                else return value;
            }
        }
        long term() {
            long value = factor();
            while (true) {
                if (take('*')) value *= factor();
                else if (take('/')) {
                    long divisor = factor();
                    if (divisor == 0) throw error("division by zero");
                    value /= divisor;
```

JAVA (continued 3/8)

```
                } else return value;
            }
        }
        long factor() {
            spaces();
            if (take('+')) return factor();
            if (take('-')) return -factor();
            if (take('(')) {
                long value = expression();
                if (!take(')')) throw error("missing ')'");
                return value;
            }
            spaces();
            if (done() || input.charAt(at) < '0' || input.charAt(at) > '9') {
                throw error("expected number");
            }
            long value = 0;
            while (!done() && input.charAt(at) >= '0' && input.charAt(at) <= '9') {
                value = Math.addExact(Math.multiplyExact(value, 10), input.charAt(at++) - '0');
            }
            return value;
        }
    }

    public static final class MinStack {
        private final Deque<Integer> values = new ArrayDeque<>();
        private final Deque<Integer> minima = new ArrayDeque<>();
        public void push(int value) {
            values.push(value);
            if (minima.isEmpty() || value <= minima.peek()) minima.push(value);
        }
        public int pop() {
            if (values.isEmpty()) throw new NoSuchElementException("empty stack");
            int value = values.pop();
            if (value == minima.peek()) minima.pop();
```

JAVA (continued 4/8)

```
            return value;
        }
        public int top() {
            if (values.isEmpty()) throw new NoSuchElementException("empty stack");
            return values.peek();
        }
        public int min() {
            if (minima.isEmpty()) throw new NoSuchElementException("empty stack");
            return minima.peek();
        }
        public boolean isEmpty() { return values.isEmpty(); }
    }

    public static final class CircularIntQueue {
        private final int[] elements;
        private int head;
        private int size;
        public CircularIntQueue(int capacity) {
            if (capacity <= 0) throw new IllegalArgumentException("capacity must be positive");
            elements = new int[capacity];
        }
        public boolean offer(int value) {
            if (size == elements.length) return false;
            elements[(head + size) % elements.length] = value;
            size++;
            return true;
        }
        public OptionalInt poll() {
            if (size == 0) return OptionalInt.empty();
            int value = elements[head];
            head = (head + 1) % elements.length;
            size--;
            return OptionalInt.of(value);
        }
```

JAVA (continued 5/8)

```
        public int size() { return size; }
    }

    public static int[] nextGreaterValues(int[] values) {
        if (values == null) throw new IllegalArgumentException("values is null");
        int[] answer = new int[values.length];
        Arrays.fill(answer, -1);
        Deque<Integer> unresolved = new ArrayDeque<>();
        for (int i = 0; i < values.length; i++) {
            while (!unresolved.isEmpty() && values[unresolved.peek()] < values[i]) {
                answer[unresolved.pop()] = values[i];
            }
            unresolved.push(i);
        }
        return answer;
    }

    public static int[] dailyTemperatures(int[] temperatures) {
        if (temperatures == null) throw new IllegalArgumentException("null input");
        int[] waits = new int[temperatures.length];
        Deque<Integer> unresolved = new ArrayDeque<>();
        for (int day = 0; day < temperatures.length; day++) {
            while (!unresolved.isEmpty()
                    && temperatures[unresolved.peek()] < temperatures[day]) {
                int older = unresolved.pop();
                waits[older] = day - older;
            }
            unresolved.push(day);
        }
        return waits;
    }

    public static long largestRectangle(int[] heights) {
        if (heights == null) throw new IllegalArgumentException("heights is null");
```

JAVA (continued 6/8)

```
        Deque<Integer> increasing = new ArrayDeque<>();
        long best = 0;
        for (int right = 0; right <= heights.length; right++) {
            int current = right == heights.length ? 0 : heights[right];
            if (current < 0) throw new IllegalArgumentException("negative height");
            while (!increasing.isEmpty() && heights[increasing.peek()] > current) {
                int height = heights[increasing.pop()];
                int left = increasing.isEmpty() ? -1 : increasing.peek();
                best = Math.max(best, (long) height * (right - left - 1));
            }
            if (right < heights.length) increasing.push(right);
        }
        return best;
    }

    public static int[] slidingMaximum(int[] values, int k) {
        if (values == null || k <= 0 || k > values.length) {
            throw new IllegalArgumentException("invalid window");
        }
        int[] answer = new int[values.length - k + 1];
        Deque<Integer> candidates = new ArrayDeque<>();
        for (int right = 0; right < values.length; right++) {
            int left = right - k + 1;
            while (!candidates.isEmpty() && candidates.peekFirst() < left) {
                candidates.removeFirst();
            }
            while (!candidates.isEmpty()
                    && values[candidates.peekLast()] <= values[right]) {
                candidates.removeLast();
            }
            candidates.addLast(right);
            if (left >= 0) answer[left] = values[candidates.peekFirst()];
        }
        return answer;
```

JAVA (continued 7/8)

```
    }

    public static int shortestGridPath(int[][] grid, int startRow, int startCol,
                                       int targetRow, int targetCol) {
        if (!inside(grid, startRow, startCol) || !inside(grid, targetRow, targetCol)
                || grid[startRow][startCol] != 0 || grid[targetRow][targetCol] != 0) {
            return -1;
        }
        boolean[][] seen = new boolean[grid.length][];
        for (int r = 0; r < grid.length; r++) seen[r] = new boolean[grid[r].length];
        ArrayDeque<int[]> queue = new ArrayDeque<>();
        queue.addLast(new int[] {startRow, startCol, 0});
        seen[startRow][startCol] = true;
        int[][] directions = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
        while (!queue.isEmpty()) {
            int[] state = queue.removeFirst();
            if (state[0] == targetRow && state[1] == targetCol) return state[2];
            for (int[] direction : directions) {
                int nr = state[0] + direction[0], nc = state[1] + direction[1];
                if (inside(grid, nr, nc) && grid[nr][nc] == 0 && !seen[nr][nc]) {
                    seen[nr][nc] = true;
                    queue.addLast(new int[] {nr, nc, state[2] + 1});
                }
            }
        }
        return -1;
    }

    private static boolean inside(int[][] grid, int row, int col) {
        return grid != null && row >= 0 && row < grid.length
                && grid[row] != null && col >= 0 && col < grid[row].length;
    }

    public static void main(String[] args) {
```

JAVA (continued 8/8)

```
        assert validDelimiters("call({x[2]})");
        assert !validDelimiters("([)]");
        assert evaluate("2 + 3 * (4 - 1)") == 11;
        assert evaluate("-(8 - 3) * 2") == -10;
        boolean nonAsciiDigitRejected = false;
        try {
            evaluate("\u0661");
        } catch (IllegalArgumentException expected) {
            nonAsciiDigitRejected = true;
        }
        assert nonAsciiDigitRejected;

        MinStack minStack = new MinStack();
        minStack.push(3); minStack.push(2); minStack.push(2);
        assert minStack.min() == 2 && minStack.pop() == 2 && minStack.min() == 2;

        CircularIntQueue queue = new CircularIntQueue(3);
        assert queue.offer(10) && queue.offer(20) && queue.offer(30);
        assert !queue.offer(99) && queue.poll().orElseThrow() == 10;
        assert queue.offer(40) && queue.size() == 3;

        assert Arrays.equals(nextGreaterValues(new int[] {2, 1, 2, 4, 3}),
                             new int[] {4, 2, 4, -1, -1});
        assert Arrays.equals(dailyTemperatures(
                new int[] {73, 74, 75, 71, 69, 72, 76, 73}),
                new int[] {1, 1, 4, 2, 1, 1, 0, 0});
        assert largestRectangle(new int[] {2, 1, 5, 6, 2, 3}) == 10;
        assert Arrays.equals(slidingMaximum(new int[] {1, 3, -1, -3, 5, 3, 6, 7}, 3),
                             new int[] {3, 3, 5, 5, 6, 7});

        int[][] grid = {{0, 0, 1}, {1, 0, 0}, {0, 0, 0}};
        assert shortestGridPath(grid, 0, 0, 2, 2) == 4;
    }
}
```

RECAP | Complexity summary

Algorithm	Time	Auxiliary space	Important qualification
delimiter validation	`O(n)`	`O(n)`	tokenization contract matters
expression parsing	`O(n)`	`O(d)`	nesting depth `d`; overflow policy separate
MinStack operation	`O(1)`	`O(n)` total	two representations form one invariant
circular queue operation	`O(1)`	`O(capacity)`	full-policy is contractual
next greater / temperatures	`O(n)`	`O(n)`	each index pushed/popped once
largest rectangle	`O(n)`	`O(n)`	use `long` for area
window maximum	`O(n)`	`O(k)`	deque stores unexpired candidate indexes
BFS	`O(V+E)`	`O(V)`	mark visited on enqueue

WATCH OUT | Edge cases and common mistakes

- Calling `pop`/`removeFirst` on empty throws; `pollFirst` returns `null`. Choose intentionally.
- `ArrayDeque` rejects null values and is neither bounded nor thread-safe.
- Delimiter counts alone cannot prove correct nesting.
- Expression evaluators need a grammar. Ad hoc "previous operator" code often mishandles unary minus and parentheses.
- `Math.negateExact(Long.MIN_VALUE)` would be needed for fully checked negation; unary minus can overflow.
- MinStack must preserve duplicate minima.
- A ring buffer with only head and tail needs another bit/slot convention to distinguish full from empty; tracking size is simpler.
- Monotonic comparisons (`<` versus `<=`) encode whether equality dominates. State the requested relation.
- Store indexes, not only values, when distance or expiration matters.
- Histogram area may overflow `int` even when each height and width fits.
- In BFS, marking on dequeue creates duplicate frontier entries; forgetting a visited rule can make cyclic graphs nonterminating.

TRY IT | Exercises with model checkpoints

Exercise 1: decode a nested repetition string

Decode input such as `3[a2[c]]`.

Checkpoint: push the prior string builder and repeat count on `[`, start a new frame, and on `]` pop and append the completed fragment the requested number of times. Validate balanced tokens, digits followed by `[`, a maximum output size, and integer overflow in counts. Complexity must include output length.

Exercise 2: postfix evaluation

Evaluate whitespace-tokenized Reverse Polish Notation.

Checkpoint: push numbers; on an operator pop right operand first, then left. Reject insufficient and leftover operands. `8 3 -` is `5`, not `−5`. Define division and overflow behavior.

Exercise 3: queue from two stacks

Implement FIFO behavior using input and output stacks.

Checkpoint: enqueue pushes to input. Dequeue transfers all input items to output only when output is empty. Each item moves at most once between stacks, proving amortized `O(1)` operations even though one dequeue can be `O(n)`.

Exercise 4: next smaller boundaries

For every histogram bar, compute nearest smaller index left and right in two passes.

Checkpoint: use consistent strict/non-strict comparisons for duplicates, then compute width. Compare memory and clarity against the one-pass sentinel solution.

Exercise 5: 0-1 BFS

Find shortest paths when edge weights are only zero or one.

Checkpoint: relax a zero-weight neighbor at the deque front and a one-weight neighbor at the back. The deque maintains nondecreasing tentative distance. Discuss why ordinary BFS fails for weight one mixed with zero and why Dijkstra is more general.

Exercise 6: bounded work queue design

Design a request-processing queue for a service whose workers can process 500 tasks per second.

Checkpoint: specify capacity in time or task units, admission timeout, overload response, cancellation, retry ownership, fairness, and metrics for depth/age/rejection. A larger queue can increase latency rather than capacity. Little's Law and measured service time guide the budget.

SDE-2 production follow-ups

How does a BFS queue relate to system backpressure? Both hold discovered work awaiting service, but graph search owns a finite known state space while production arrivals may be unbounded. A service needs capacity and an explicit overload signal to upstream callers.

Can `ArrayDeque` be shared between threads? Not safely without external synchronization. Choose a concurrent queue whose semantics match the operation; even then, compound checks such as "if absent then enqueue" may need stronger coordination.

How do you make parsers robust? Cap input length and nesting, return structured error positions, avoid quadratic substring construction, define Unicode/token rules, and use exact arithmetic or a documented overflow policy. Fuzz malformed inputs and ensure failure consumes bounded time and memory.

What should be observable? Queue depth, oldest-item age, enqueue/dequeue rate, rejection count, service latency, timeout/cancellation rate, and saturation duration. For algorithmic monotonic structures, instrument input size and operation count when investigating performance; aggregate linearity should be visible.

Interview follow-up chain and model answers

Why does a nested `while` in a monotonic stack remain linear? Charge work to entries, not loop nesting. Every index is pushed exactly once. Once popped, it never re-enters. Across the entire scan there are at most `n` pushes and `n` pops, so total deque operations are `O(n)` even if one input element triggers many pops.

When should equal values be removed from a monotonic deque? For window maximum, removing an older equal value is safe because the newer equal value produces the same maximum and expires later. This keeps the deque smaller. If the contract asks for the earliest index of a maximum, retain equals instead. The comparison embodies a tie-breaking contract, not only a micro-optimization.

Why mark BFS vertices when enqueued? Enqueueing is the moment a shortest layer first discovers the vertex. Marking then prevents every other same/next-layer parent from adding duplicates. Marking only on removal may preserve distance correctness but can inflate the queue dramatically, and without careful handling can repeat work.

What makes queue backpressure different from a lock? A lock coordinates exclusive access to state. Backpressure limits outstanding work and signals that downstream capacity is exhausted. A thread-safe unbounded queue solves race conditions but can still consume memory until failure. A bounded queue needs a full policy that propagates overload rather than merely hiding it.

How would you make MinStack persistent? Store immutable nodes containing value, minimum-so-far, and previous. Each push returns a new head, and pop returns the previous head, so versions share tails safely. Operations remain `O(1)` and snapshots are cheap, at the cost of one allocation per push and eventual garbage collection.

Can sliding-window maximum handle an online stream? Yes, if each item has a monotonically increasing sequence or timestamp. Evict front candidates outside the current window and remove dominated backs. Time windows also need a policy for late/out-of-order events and watermark progress; count windows do not. A production operator should expose state size, lag, and late-event counts.

A complete pattern-selection checkpoint

When a prompt says "nearest," do not immediately choose a heap. Ask whether the answer must respect sequence position. A heap retrieves a global extreme but does not efficiently discard an arbitrary expired element. A monotonic deque retains only nondominated candidates in positional order. When a prompt says "minimum number of transitions," inspect edge weights: FIFO BFS proves shortest paths only when every edge costs the same; 0-1 BFS uses a deque, and general nonnegative weights require a priority queue. When a prompt says "process tasks," separate algorithmic order from operational policy: FIFO may be fair enough for a finite graph, but a service may need priorities, tenant fairness, deadlines, idempotency, and poison-task handling. This selection conversation demonstrates more senior judgment than reciting container APIs.

Final readiness checklist

- I choose LIFO, FIFO, or two-ended access from the ordering requirement.
- I can use `ArrayDeque` without mixing front and back conventions.
- My stack/queue invariant explains every stored entry.
- I prove aggregate linear monotonic work by push/pop counts.
- I store indexes when age, distance, or expiration matters.
- BFS marks on enqueue and states the unweighted-edge assumption.
- Capacity, overload, cancellation, and concurrency are explicit production policies.

The transferable skill is not knowing that a deque exists. It is seeing which unresolved states deserve to remain and proving when an old state can never matter again.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: stack and queue simulations
- Interview Core: parentheses, next greater, and window maximum
- SDE-2 Follow-up: bounded queues and API contracts
- Challenge: histogram and monotonic-deque variants

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous volume: 10 - Linked Lists: Pointer Reasoning and Mutation

Series index

Next volume: 12 - Binary Search: Bounds, Answers, and Invariants

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 11 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.